



EXAMEN SEGUNDO CORTE - PROGRAMACIÓN ORIENTADA A OBJETOS
SISTEMA DE GESTIÓN DE AEROLÍNEA FLUXIOAIR
TIEMPO: 2 HORAS

Docente: Ing. Esp. Amilkar Hernandez

Asignatura: Programación de Computadores II (SS300)

Tipo: Parcial Practico – Segundo Corte

Modalidad: Grupo 2 Máximo

CONTEXTO DEL PROBLEMA:

La aerolínea "**FLUXIOAIR**" es una compañía aérea que opera vuelos nacionales e internacionales en América Latina. Actualmente, la gestión de sus operaciones se realiza de forma manual y desorganizada, lo que ha generado problemas como:

- Pérdida de información de pasajeros
- Reservas duplicadas o perdidas
- Dificultad para conocer la disponibilidad de asientos en tiempo real
- Imposibilidad de calcular correctamente los ingresos totales

FluxioAir te ha contratado como desarrollador de software para crear un sistema de gestión integral que automatice sus procesos y resuelva estos problemas.

OBJETIVO DEL SISTEMA:

Desarrollar una aplicación de consola en Java que permita a FluxioAir:

1. Gestionar la información de sus pasajeros
2. Administrar su catálogo de vuelos (nacionales e internacionales)
3. Procesar reservas de boletos aéreos
4. Generar reportes y estadísticas de operación

DESCRIPCIÓN DETALLADA DEL NEGOCIO:

1. GESTIÓN DE PASAJEROS:

FluxioAir atiende a pasajeros que desean viajar a diferentes destinos. Cada pasajero que utiliza los servicios de la aerolínea debe estar registrado en el sistema.

Información que se requiere de cada pasajero:

- Número de cédula (único e irrepensible)
- Nombres
- Apellidos
- Edad (debe ser mayor o igual a 0 años, ya que viajan bebés)



- Correo electrónico
- Número de teléfono de contacto
- Número de pasaporte (único, necesario para viajes)
- Nacionalidad (país de origen del pasajero)

Reglas de negocio:

- No pueden existir dos pasajeros con la misma cédula
- No pueden existir dos pasajeros con el mismo número de pasaporte
- El correo electrónico debe contener el símbolo "@" utilizar el metodo contains()

2. GESTIÓN DE VUELOS:

FluxioAir opera dos tipos de vuelos: Nacionales e Internacionales.

Información común de todos los vuelos:

- Código de vuelo (único, ejemplo: "GA001", "GA002")
- Ciudad de origen
- Ciudad de destino
- Fecha del vuelo (formato: DD/MM/YYYY)
- Hora de salida (formato: HH:MM, ejemplo: "14:30")
- Hora de llegada (formato: HH:MM, ejemplo: "16:45")
- Capacidad total de asientos del avión
- Asientos disponibles (inicialmente igual a la capacidad total)
- Precio base del boleto
- Estado del vuelo: puede ser "Programado", "En Vuelo", "Aterrizado" o "Cancelado"

Vuelos Nacionales:

Son vuelos que operan dentro del mismo país (Colombia). Además de la información común, tienen:

- Duración estimada en horas (ejemplo: 1.5 horas, 2 horas)
- Incluye alimentación (Sí o No)

Cálculo del precio final:

- El precio final es igual al precio base (sin cargos adicionales)

Ejemplos de rutas nacionales:

- Bogotá → Cartagena
- Medellín → Cali
- Barranquilla → San Andrés
- Cali → Bogotá

Vuelos Internacionales:

Son vuelos que viajan a otros países. Además de la información común, tienen:

- País de destino
- Requiere visa (Sí o No - según el país destino)



- Cargo internacional (valor adicional por impuestos internacionales, ejemplo: \$50,000)

Cálculo del precio final:

- El precio final es igual al precio base + cargo internacional

Ejemplos de rutas internacionales:

- Bogotá → Miami (USA) - Requiere visa: Sí
- Cartagena → Panamá (Panamá) - Requiere visa: No
- Medellín → Madrid (España) - Requiere visa: Sí
- Cali → Lima (Perú) - Requiere visa: No

Reglas de negocio:

- No pueden existir dos vuelos con el mismo código
- Los asientos disponibles nunca pueden ser negativos
- La capacidad total debe ser mayor a 0
- El precio base debe ser mayor a 0

3. SISTEMA DE RESERVAS:

Los pasajeros realizan reservas para viajar en los vuelos de FluxioAir. Una reserva es el proceso mediante el cual un pasajero adquiere uno o más boletos para un vuelo específico.

Información de cada reserva:

- Código de reserva (único, ejemplo: "R001", "R002")
- Pasajero que realiza la reserva
- Codigo Vuelo reservado (nacional o internacional)
- Cantidad de asientos reservados (mínimo 1, máximo 5 por reserva)
- Fecha en que se realizó la reserva
- Precio total de la reserva
- Estado de la reserva: puede ser "Confirmada", "Cancelada" o "Completada"

Proceso de creación de una reserva:

1. El sistema debe verificar que el pasajero existe en el sistema
2. El sistema debe verificar que el vuelo existe y está en estado "Programado"
3. El sistema debe verificar que hay suficientes asientos disponibles
4. Si todo es correcto:
 - Se crea la reserva con estado "Confirmada"
 - Se reducen los asientos disponibles del vuelo
 - Se calcula el precio total: precio del vuelo × cantidad de asientos

Proceso de cancelación de una reserva:

1. Se busca la reserva por su código
2. Se cambia el estado a "Cancelada"
3. Se devuelven los asientos al vuelo (se aumentan los asientos disponibles)

Reglas de negocio:



- No se puede reservar en un vuelo que no esté en estado "Programado"
- No se puede reservar más asientos de los disponibles
- Una reserva solo puede tener entre 1 y 5 asientos
- Una vez confirmada una reserva, el pasajero puede cancelarla
- No se pueden crear dos reservas con el mismo código

4. ESTADÍSTICAS Y REPORTES REQUERIDOS:

El sistema debe permitir consultar la siguiente información:

1. Reservas por código: Consultar la reserva por código y mostrar toda su información.
2. Total de pasajeros registrados
3. Reservas por pasajero: Consultar todas las reservas de un pasajero específico

5. CASOS DE USO DEL SISTEMA:

Caso 1: Registro de un nuevo pasajero

El usuario del sistema ingresa:

- Cédula: 1234567890
- Nombre: Juan
- Apellido: Pérez
- Edad: 35
- Email: juan.perez@email.com
- Teléfono: 3001234567
- Pasaporte: P12345678
- Nacionalidad: colombiana

El sistema valida la información y registra al pasajero.

Caso 2: Crear un vuelo internacional

El administrador crea un nuevo vuelo:

- Código: GA100
- Origen: Bogotá
- Destino: Miami
- Fecha: 15/12/2024
- Hora salida: 08:00
- Hora llegada: 13:00
- Capacidad: 180 asientos
- Precio base: \$800,000
- País destino: Estados Unidos
- Requiere visa: Sí
- Cargo internacional: \$100,000 Precio final calculado: \$900,000

Caso 3: Realizar una reserva

- Un pasajero desea reservar:



- Cédula del pasajero: 1234567890
- Código de vuelo: GA100
- Cantidad de asientos: 2

El sistema:

1. Busca al pasajero por cédula (debe existir)
2. Busca el vuelo por código (debe existir y estar programado)
3. Verifica que hay 2 asientos disponibles (180 disponibles)
4. Crea la reserva con código R001
5. Calcula precio total: $\$900,000 \times 2 = \$1,800,000$
6. Reduce asientos disponibles del vuelo: $180 - 2 = 178$
7. Muestra confirmación al usuario

Caso 5: Cancelar una reserva

- El pasajero desea cancelar la reserva R001:
- Se busca la reserva
- Se cambia estado a "Cancelada"
- Se devuelven los 2 asientos al vuelo GA100
- Asientos disponibles: $178 + 2 = 180$

SITUACIONES EXCEPCIONALES:

Situación	Mensaje esperado
Cédula duplicada	"Ya existe un pasajero con esa cédula"
Sin asientos disponibles	"No hay asientos disponibles para este vuelo"
Más de 5 asientos	"No se pueden reservar más de 5 asientos por reserva"
Precio base negativo o cero	"El precio del vuelo debe ser mayor a cero"
Reserva no encontrada	"No se encontró la reserva con ese código"
Pasajero no existe al reservar	"No se encontró el pasajero con esa cédula"
Email sin @	"El email debe contener el símbolo @"

Conceptos que DEBE aplicar:

1. Herencia:
 - Identificar jerarquías de clases (clases padre e hijas)
2. ArrayList:
 - Para almacenar colecciones de objetos en memoria



3. Relaciones entre clases:
 - Una clase que usa objetos de otra clase
6. Encapsulamiento:
 - Atributos privados/protected con getters y setters

ENTREGABLES:

1. Análisis del problema:
 - Identificar las clases necesarias del modelo
 - Identificar las jerarquías de herencia
 - Identificar las relaciones entre clases
2. Diagrama UML:
 - Diagrama de clases completo del sistema
 - Debe incluir: atributos, métodos, relaciones, herencia
3. Implementación en Java:
 - Código fuente completo y funcional
 - Organizado en la estructura de capas requerida
 - Todos los métodos implementados
 - Manejo de todas las validaciones
4. Demostración:
 - El sistema debe ejecutarse sin errores
 - Debe permitir realizar todas las operaciones descritas
 - Debe manejar correctamente las situaciones excepcionales

